Geodesica: Real-Time Schwarzschild Black Hole Rendering via GPU-Side Geodesic Integration

Aanish Farrukh
Independent / FarrukhCorp

March 9, 2026

Abstract

Geodesica is a real-time black hole renderer written in TypeScript, WebGL2, and GLSL ES 3.00. The simulator visualizes a Schwarzschild black hole by integrating null geodesics per pixel directly inside a fragment shader, rather than relying on precomputed lookup textures or offline ray tracing. Its core numerical method is fourth-order Runge–Kutta (RK4) integration of an equivalent Cartesian form of the Schwarzschild photon orbit equation, allowing the program to produce gravitational lensing, a photon ring, multi-crossing accretion-disk accumulation, gravitational redshift, relativistic Doppler amplification, and observer aberration in real time. The rendering pipeline is organized as a two-pass system: a high-dynamic-range geodesic pass writes to a floating-point framebuffer, followed by bloom, ACES filmic tone mapping, and vignette post-processing. This paper documents the physical model actually implemented in the simulator, the numerical approximations required for interactive performance, and the places where the renderer intentionally simplifies real black hole astrophysics in order to remain responsive in the browser.

1 Introduction

The Schwarzschild solution is the simplest exact black hole spacetime in general relativity: static, spherically symmetric, and non-rotating. Even this "minimal" black hole produces visually rich optical effects, including strong gravitational lensing, a bright photon ring, and dramatic distortions of a surrounding accretion flow.

Black hole visualization has a long history. Luminet's 1979 paper produced one of the first physically motivated images of a black hole with a thin accretion disk [2]. More recently, the rendering work behind *Interstellar* demonstrated how cinematic black hole imagery could still be grounded in relativistic ray tracing [3]. Web-based real-time visualizers such as `schwarzschild` by rantonels [5] and `black-hole` by oseiskar [6] showed that browser GPU pipelines can support interactive black hole rendering.

Geodesica differs from offline renderers by integrating geodesics on the GPU for every visible pixel at interactive frame rates. Rather than tracing rays on the CPU or relying on precomputed lensing maps, the simulator runs a fragment-shader RK4 integrator in real

time. That design keeps the system interactive while still preserving the dominant visual signatures of Schwarzschild spacetime.

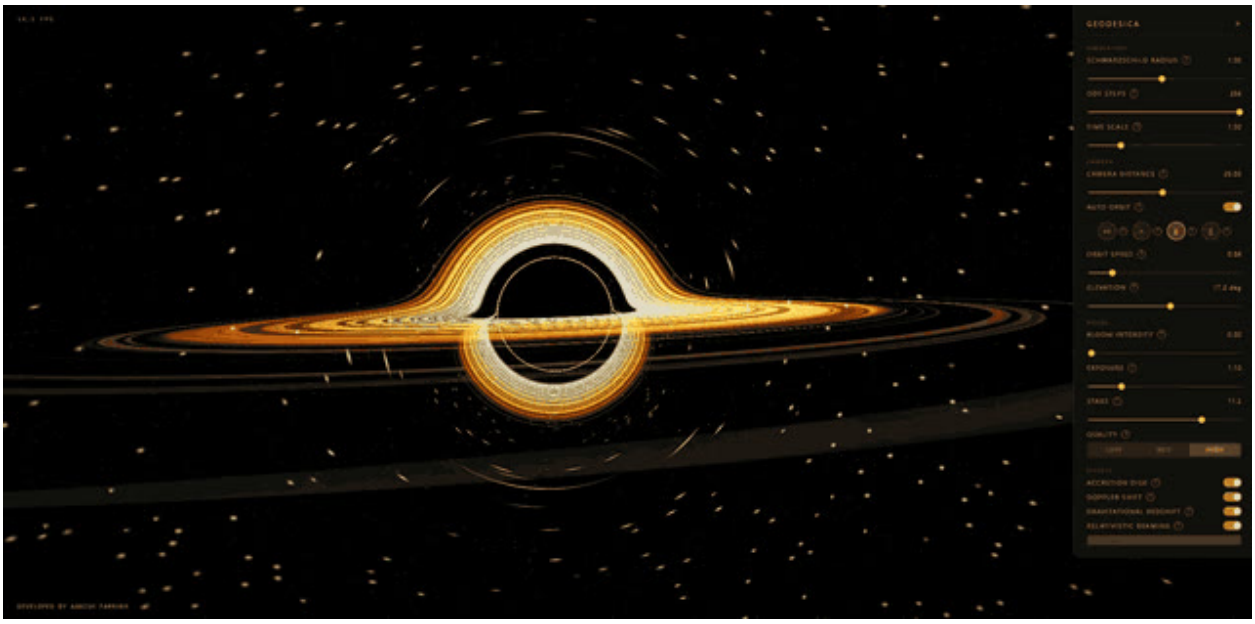

Figure 1: A representative frame from Geodesica showing the lensed accretion disk, photon ring, procedural star background, and interactive control panel.

2 The Schwarzschild Spacetime

In geometric units, the Schwarzschild metric is

$$ds^2 = -\left(1 - \frac{r_s}{r}\right) dt^2 + \left(1 - \frac{r_s}{r}\right)^{-1} dr^2 + r^2 d\Omega^2, \tag{1}$$

where

$$r_s = \frac{2GM}{c^2} \tag{2}$$

is the Schwarzschild radius. The event horizon lies at

$$r = r_s, \tag{3}$$

the photon sphere lies at

$$r = \frac{3}{2} r_s, \tag{4}$$

and the innermost stable circular orbit (ISCO) for massive particles lies at

$$r = 3r_s. \tag{5}$$

Geodesica exposes r_s as a user control and uses it as the master scale for the event horizon, the photon-ring highlighting term, and the disk geometry. The disk itself is rendered from $3r_s$ outward, matching the ISCO-based inner edge used in the shader.

3 Null Geodesics and the Photon Orbit Equation

For null geodesics in Schwarzschild spacetime, the conserved energy E and angular momentum L give the effective radial equation

$$\left(\frac{\mathrm{d}r}{\mathrm{d}\lambda}\right)^2 + V_{\text{eff}}(r) = E^2, \tag{6}$$

with photon effective potential

$$V_{\text{eff}}(r) = \left(1 - \frac{r_s}{r}\right)\frac{L^2}{r^2}. \tag{7}$$

Introducing $u = 1/r$ and using the azimuthal angle ϕ as the orbit parameter yields the standard photon orbit equation

$$\frac{\mathrm{d}^2 u}{\mathrm{d}\phi^2} + u = \frac{3}{2} r_s u^2, \tag{8}$$

which appears as Eq. 25.40 in Misner, Thorne, and Wheeler [1].

Geodesica does *not* integrate Eq. (8) directly in (u, ϕ) form. Instead, the fragment shader uses an equivalent Cartesian central-force form derived from the same Schwarzschild correction:

$$\frac{d^2\mathbf{r}}{d\lambda^2} = -\frac{3}{2} r_s h^2 \frac{\mathbf{r}}{\|\mathbf{r}\|^5}, \tag{9}$$

where $h^2 = \|\mathbf{r} \times \mathbf{v}\|^2$ is the squared specific angular momentum of the ray. This form is convenient for GPU execution because each pixel can evolve a ray state $(\mathbf{r}, \mathbf{v})$ without solving coordinate singularities in angular variables.

4 Numerical Integration: RK4 on the GPU

Explicit Euler integration is cheap, but it accumulates visible trajectory error rapidly in strong-curvature regions near the photon sphere. Geodesica therefore uses classical fourth-order Runge–Kutta integration. For a state $\mathbf{y}$ satisfying $\dot{\mathbf{y}} = f(\mathbf{y})$, RK4 updates via

$$\mathbf{k}_1 = f(\mathbf{y}_n), \tag{10}$$

$$\mathbf{k}_2 = f\left(\mathbf{y}_n + \frac{\Delta t}{2}\mathbf{k}_1\right), \tag{11}$$

$$\mathbf{k}_3 = f\left(\mathbf{y}_n + \frac{\Delta t}{2}\mathbf{k}_2\right), \tag{12}$$

$$\mathbf{k}_4 = f(\mathbf{y}_n + \Delta t\, \mathbf{k}_3), \tag{13}$$

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{\Delta t}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right). \tag{14}$$

In Geodesica, every fragment stores the ray position and direction in a `RayState` struct and advances it in the fragment shader. The current implementation allows 32 to 256 user-selected ODE steps, with quality presets mapped to 64, 128, and 256 steps. The renderer

also scales step size with both camera distance and quality preset, so higher quality means more steps and smaller steps.

The actual GLSL integrator is compact enough to run directly per pixel:

```
RayDeriv geodesicDeriv(RayState state) {
  float r = max(length(state.pos), EPSILON);
  vec3 angularMomentum = cross(state.pos, state.vel);
  float h2 = dot(angularMomentum, angularMomentum);
  float r2 = r * r;
  float r5 = max(r2 * r2 * r, EPSILON);
  float coeff = -1.5 * uSchwarzschildRadius * h2 / r5;
  RayDeriv deriv;
  deriv.dPos = state.vel;
  deriv.dVel = state.pos * coeff;
  return deriv;
}

RayState rk4Step(RayState state, float dt) {
  RayDeriv k1 = geodesicDeriv(state);
  RayDeriv k2 = geodesicDeriv(applyDeriv(state, k1, dt * 0.5));
  RayDeriv k3 = geodesicDeriv(applyDeriv(state, k2, dt * 0.5));
  RayDeriv k4 = geodesicDeriv(applyDeriv(state, k3, dt));

  state.pos += (k1.dPos + 2.0 * k2.dPos + 2.0 * k3.dPos + k4.dPos) *
      (dt / 6.0);
  state.vel += (k1.dVel + 2.0 * k2.dVel + 2.0 * k3.dVel + k4.dVel) *
      (dt / 6.0);
  state.vel = normalize(state.vel);
  return state;
}
```

Because this work happens inside the fragment shader, each output pixel independently ray-marches through the Schwarzschild field. This is the core reason the simulator remains interactive: the browser GPU runs thousands of geodesic integrations in parallel.

5 The Accretion Disk

Thin-disk theory provides the astrophysical backdrop for Geodesica's disk model. In the Shakura–Sunyaev picture, the effective temperature scales approximately as

$$T(r) \propto \left[\frac{1}{r^3}\left(1-\sqrt{\frac{r_i}{r}}\right)\right]^{1/4}, \tag{15}$$

with r_i the inner edge of the disk [4]. A more complete radiative transfer model would map this temperature to color using a blackbody spectrum.

Geodesica uses that theory as inspiration, but the implemented shader is intentionally simpler. The disk is represented as a geometrically thin tilted annulus with inner radius

$3r_s$ and outer body radius $14r_s$, plus spiral-arm structure extending to $22r_s$. Rather than evaluating Eq. (15) explicitly, the shader uses a hand-tuned radial color gradient, vertical density falloff, and procedural modulation terms:

- a radial emissivity ramp from bright yellow-white near the inner disk to dark red-brown farther out,
- fractal Brownian motion (FBM) for fiber-like turbulent texture,
- a spiral-arm modulation for coherent rotating structure,
- exponential vertical attenuation to keep the disk thin,
- explicit annulus masking to preserve the disk silhouette.

Differential rotation is encoded by phase terms proportional to

$$\omega(r) \propto r^{-3/2}, \tag{16}$$

which matches the Keplerian scaling used in thin-disk intuition. When a ray crosses the tilted disk plane multiple times, Geodesica accumulates contributions from each intersection, reducing transparency by a fixed factor after each hit. That gives the disk visible depth without resorting to volumetric simulation.

6 Relativistic Visual Effects

6.1 Gravitational Lensing

The lensing effect in Geodesica emerges directly from the RK4 geodesic integration. Each fragment launches a ray from the camera, bends it according to Eq. (9), and then samples the background using the final deflected direction. Strong lensing appears automatically near the hole, while the bright photon ring is emphasized with an additional glow term based on periapsis distance to the photon sphere:

$$I_{\text{ring}} \propto \exp\left[-\frac{(r_{\min} - 1.5r_s)^2}{\sigma^2}\right]. \tag{17}$$

6.2 Doppler Beaming

For disk emission, the shader uses the relativistic Doppler factor

$$D = \sqrt{\frac{1+\beta}{1-\beta}}, \tag{18}$$

where β is the line-of-sight velocity in units of c. Disk intensity is scaled approximately as

$$I \propto D^4, \tag{19}$$

which strongly brightens the approaching side and dims the receding side. The orbital speed approximation used in the shader is

$$\frac{v}{c} \approx \sqrt{\frac{r_s}{2r}}. \tag{20}$$

This is the dominant reason one side of the disk appears dramatically brighter.

6.3 Gravitational Redshift

Gravitational redshift is modeled using

$$1 + z = \frac{1}{\sqrt{1 - r_s/r}}. \tag{21}$$

In practice, Geodesica converts this into a scalar dimming and a warmer color balance near the hole. The effect is intentionally simplified: it is not a full spectral transport calculation, but it does convey the expected visual darkening and reddening of emission close to the horizon.

6.4 Relativistic Aberration

The simulator also supports an observer-motion mode. The scene controller estimates observer velocity from keyboard-driven camera translation, and the fragment shader applies an aberration transform to the outgoing ray direction before tracing. This shifts the apparent sky position of stars and affects the brightness of the sampled background, producing a plausible motion-induced relativistic distortion.

7 Rendering Pipeline

Geodesica targets WebGL2 with GLSL ES 3.00. The main reasons for choosing WebGL2 over WebGL1 are support for modern shader syntax and floating-point render targets through `EXT_color_buffer_float`.

The renderer uses a two-pass pipeline:

1. A geodesic pass traces the black hole scene into an `RGBA16F` framebuffer.
2. A post-processing pass applies bloom, exposure, ACES filmic tone mapping, gamma correction, and vignette before presenting to the canvas.

Bloom is implemented with a bright-pass threshold and a compact multi-tap blur at two radii. Tone mapping uses the ACES filmic curve,

$$\mathrm{ACES}(x) = \mathrm{clamp}\left(\frac{x(2.51x + 0.03)}{x(2.43x + 0.59) + 0.14}, 0, 1\right), \tag{22}$$

which compresses the HDR image into display space while preserving highlight structure better than a linear exposure alone.

The background star field is also procedural. No external texture skybox is used in the current simulator; instead, multiple star layers are synthesized with hashed spherical UV cells and simple color-category selection in the shader.

8 Where the Simulation Simplifies Reality

Several simplifications are deliberate:

- Only the Schwarzschild metric is implemented; there is no Kerr spin term.
- The accretion disk is geometrically thin and does not model full vertical structure.
- Disk turbulence is procedural FBM, not magnetohydrodynamic simulation.
- Disk color is a stylized emissivity ramp, not a full blackbody spectrum computed from Eq. (15).
- Background stellar spectra are approximated procedurally.
- Lower ODE step counts and larger step sizes introduce visible lensing error.
- Polarization is not modeled.
- The renderer is monochromatic in the sense of display RGB only; it does not simulate multi-wavelength channels such as X-ray or radio bands.
- The photon ring is enhanced with an explicit glow term for readability.

These simplifications are acceptable for an interactive browser renderer, but they should not be confused with a high-fidelity astrophysical transport code.

9 Performance

The fragment-shader RK4 integration dominates frame time. That cost scales roughly linearly with the number of ODE steps, and in Geodesica the quality presets also change step size, making the high preset both more accurate and more expensive. The browser renderer further limits device pixel ratio to 2 in order to avoid runaway fill-rate cost on high-density displays.

Table 1 gives illustrative step-count scaling. These values are not hard benchmarks baked into the repository; they are approximate desktop-class estimates meant to communicate the trade-off between speed and fidelity.

Table 1: Illustrative step-count scaling in Geodesica.

Steps	Approx. frame time	Photon ring accuracy
32	∼2 ms	Poor
64	∼4 ms	Acceptable
128	∼8 ms	Good
256	∼16 ms	Excellent

The quality preset system maps directly onto the user interface:

- `low`: 64 steps,
- `med`: 128 steps,
- `high`: 256 steps.

On mobile hardware, reducing steps, bloom intensity, and star density is the most direct way to preserve responsiveness.

10 Conclusion

Geodesica demonstrates that a browser-based renderer can produce convincing Schwarzschild black hole imagery in real time without precomputed textures or offline ray tracing. Its core contribution is straightforward but effective: each fragment integrates a null geodesic using RK4 on the GPU, then shades the disk and background from the resulting trajectory. On top of that core, the simulator layers Doppler boosting, gravitational redshift, observer aberration, HDR post-processing, and interactive camera controls.

Compared with prior real-time browser projects such as rantonels' `schwarzschild` [5] and oseiskar's `black-hole` [6], Geodesica emphasizes a self-contained procedural pipeline with a stylized but physically motivated disk and explicitly documented implementation trade-offs. Natural future directions include Kerr geodesics, polarization, more faithful spectral transport, and disk models informed by MHD rather than procedural turbulence.

References

[1] Misner, C. W., Thorne, K. S., Wheeler, J. A. (1973). *Gravitation*. W. H. Freeman.

[2] Luminet, J.-P. (1979). Image of a spherical black hole with thin accretion disk. *Astronomy and Astrophysics*, 75, 228–235.

[3] James, O., von Tunzelmann, E., Franklin, P., Thorne, K. S. (2015). Gravitational lensing by spinning black holes in astrophysics, and in the movie *Interstellar*. *Classical and Quantum Gravity*, 32(6).

[4] Shakura, N. I., Sunyaev, R. A. (1973). Black holes in binary systems. Observational appearance. *Astronomy and Astrophysics*, 24, 337–355.

[5] rantonels. (2015). *schwarzschild: A real-time WebGL visualization of a Schwarzschild Black Hole*. GitHub. https://github.com/rantonels/schwarzschild

[6] oseiskar. (2016). *black-hole: WebGL simulation of a Schwarzschild black hole*. GitHub. https://github.com/oseiskar/black-hole